

High-Performance Implementation of Multigrid Solver for Lattice QCD

10 分钟汇报完整讲稿及补充参考

张鑫

中国科学院近代物理研究所

2026 年 10 月 11 日

摘要

本讲稿对应 18 页横版汇报稿。每页包含三部分：可直接朗读的主口播稿、用于加深理解的公式或数据说明，以及时间允许或回答提问时使用的补充参考。主稿控制在约 10 分钟；补充内容不属于必须逐字朗读的部分，主要用于保证论证完整、上下文清楚和问答可控。所有数值以 `git tag test27` 为正式性能基准，508 报告单独作为早期大规模测试成果。

时间与内容控制

页码	主题	主稿时长	时间不足时保留
1	标题与范围	15 s	题目、汇报人、test27 基线
2	Wilson 与 Clover	35 s	D_W 、Clover onsite block
3	原始 Dslash	30 s	裸核 H 、八个邻居、带宽瓶颈
4	奇偶 Schur	35 s	Schur 补、求解、重建
5	MultiGrid 构造	35 s	null vector、P/R、Galerkin
6	预条件 Bi-CGStab	40 s	$\hat{p}$ 、 v 、 s 、 ω 更新
7	Strict 特化	30 s	$D = X + H$ 与运行资产
8	粗层 X/Y	35 s	X/Y 定义、 $\hat{Y}$ 顺序
9	V-cycle 与 FGMRES	40 s	内外层关系、真残差刷新
10	CUDA-C++ 优化	40 s	带宽、延迟、setup、通信
11	QUDA 对照口径	30 s	66/132、1+2+5、trace-off
12	总体结果	35 s	35/44、21/22、0/22
13	设备与精度	35 s	V100/P100 分组中位数
14	阶段与显存	40 s	coarse+other、容量替代
15	残差与下一步	35 s	5×10^{-6} 门限、优化方向
16	复现与文献	20 s	最小闸门、参考实现
17	508 大规模成果	25 s	TEST8 c64 64–1024 进程、3.12/7.19
18	致谢	10 s	一个结论、一个行动项

建议口播总量约 520–560 秒，留出约 60–80 秒用于翻页、强调和停顿。补充参考部分逐页紧随主稿，不需要全部朗读。

1 逐页主稿与补充参考

1.1 第 1 页 | 标题与汇报范围（主稿约 15 秒）

主口播稿 各位老师、各位同行，大家好。我汇报的题目是面向格点 QCD 传播子求解的高性能多重网格实现。这里重点讲 PyQCU 的 MultiGrid 求解器以及它的 CUDA-C++ 核心，不展开整个项目历史。性能结论统一采用 `git tag test27`，旧标签的结果只用于解释实现演进，不混入最终对照。

补充参考 本次汇报的核心问题是：在相同格点、精度、求解器、初始猜测和真残差门下，PyQCU 的 MultiGrid 与 QUDA 相比处于什么位置，优势来自哪里，限制又是什么。汇报主线是“传播子算子 → 多重网格算法 → CUDA-C++ 实现 → QUDA 对照 → 容量与下一步”。项目地址、文档和 tag 日志均可在标题页直接访问。当前版本是研究实现，不宣称所有求解器或所有硬件场景普遍领先。

1.2 第 2 页 | Wilson 与 Clover 算子（主稿约 35 秒）

主口播稿 先固定细网格算子。Wilson 的裸跃迁核由四个正向和四个反向 neighbors 组成。物理归一化 Wilson 算子是 $D_W = (m_0 + 4)I - \frac{1}{2}H$ ，也可写成 $D_W = (m_0 + 4)(I - \kappa H)$ ，其中 $\kappa = 1/(2m_0 + 8)$ ，因此 $m_0 + 4 = 1/(2\kappa)$ 。Clover 改进在相同最近邻结构上增加一个 onsite 12×12 spin-color 块 $A_p = I_p + T_p$ 。

补充参考 设规范链接为 $U_{x,\mu} \in SU(3)$ ，则

$$(H\psi)_x = \sum_{\mu=0}^3 \left[(1 - \gamma_\mu) U_{x,\mu} \psi_{x+\hat{\mu}} + (1 + \gamma_\mu) U_{x-\hat{\mu},\mu}^\dagger \psi_{x-\hat{\mu}} \right]. \quad (1)$$

Clover 项由有向四点规范环构造：

$$C_{\mu\nu} = Q_{\mu\nu} - Q_{\mu\nu}^\dagger, \quad T_p = -\frac{\kappa c_{sw}}{8u_0} \sum_{\mu < \nu} \gamma_\mu \gamma_\nu C_{\mu\nu}. \quad (2)$$

当前 C++ 生产路径取 $u_0 = 1$ ，有效 $c_{sw} = 1$ 。需要特别注意：原始 Dslash 入口返回裸核 H ，不含单位项和 $-\kappa$ ；带单位项的入口返回 $I - (\kappa/u_0)H$ 。若与 QUDA 或其他代码比较，归一化差异必须先消除。

1.3 第 3 页 | 原始 Dslash（主稿约 30 秒）

主口播稿 Dslash 是矩阵自由的最近邻稀疏作用。每个格点读取八个邻居；对每个方向先做 $1 \pm \gamma_\mu$ 投影，再执行规范平移和 $SU(3)$ 乘法。它天然 memory/bandwidth-bound。PyQCU 把四分量 spin projection 降到两个独立分量，并利用 $SU(3)$ 结构和尾置时空维提高访存效率。

补充参考 本页最重要的是对象区分：

- `applyWilsonDslashQcu` 返回 H ；
- `give_wilson_eo/oe` 返回 $-(\kappa/u_0)H_{pq}$ ；

- `give_wilson(..., with_I=True)` 返回 $I - (\kappa/u_0)H$;
- `applyCloverDslashQcu` 返回 $A_p^{-1}H_{pq}$ 。

Dslash kernel 的优化重点不是减少邻居数，而是让 spin projector、规范链接读取和输出累加形成连续数据流。C++ 路径按 parity 半体积执行，并把 send、inside、receive 和 correction 分阶段，为 MPI halo 做准备。

1.4 第 4 页 | 奇偶 Schur 消元 (主稿约 35 秒)

主口播稿 为了减小外层 Krylov 维度，PyQCU 将场分成 even 和 odd 两部分。以奇数子格为例，精确消去 even 分量后得到 Schur 补 $S_o = A_o - \kappa^2 H_{oe} A_e^{-1} H_{eo}$ 。求得 x_o 后，再用 $x_e = A_e^{-1}(b_e + \kappa H_{eo} x_o)$ 精确重建完整解。

补充参考 完整块方程可写为

$$\begin{pmatrix} A_e & -\kappa H_{eo} \\ -\kappa H_{oe} & A_o \end{pmatrix} \begin{pmatrix} x_e \\ x_o \end{pmatrix} = \begin{pmatrix} b_e \\ b_o \end{pmatrix}. \quad (3)$$

消元后的右端为

$$b_o^{\text{pc}} = b_o + \kappa H_{oe} A_e^{-1} b_e. \quad (4)$$

Wilson 情形下 $A_e = A_o = I$ ，所以 $S_o = I - \kappa^2 H_{oe} H_{eo}$ 。Schur 求解只改变未知量组织，不改变原方程；最终收敛必须由完整 Wilson/Clover 算子的真残差复位，续推残差不能代替真残差。

1.5 第 5 页 | MultiGrid 构造 (主稿约 35 秒)

主口播稿 MultiGrid 把误差分成高频和低频。平滑器处理高频误差，粗空间表示低频误差。每层先通过 $A_l B_l \approx 0$ 生成近零模，再在 aggregate 内做局部正交化得到 P_l ，并令 $R_l = P_l^\dagger$ 。粗算子通过 Galerkin 投影 $A_{l+1} = R_l A_l P_l$ 构造。

补充参考 B_l 只是低模近似空间，不是精确核。局部正交满足 $P_l^\dagger P_l = I$ ，但 $P_l P_l^\dagger \neq I$ 。因此粗空间不能替代细层单位算子。aggregate 的支撑互不重叠，限制过程不需要全局 Gram 逆；同时 Galerkin 后的算子仍保持有限支撑，可以继续递归粗化。

1.6 第 6 页 | 预条件 Bi-CGStab (主稿约 40 秒)

主口播稿 这一页给出粗层 Bi-CGStab 的完整算法。初始化残差后，每轮先计算 ρ_{i-1} ，构造搜索方向 $p^{(i)}$ ；再通过预条件器求 $\hat{p}$ ，应用原算子得到 $v^{(i)}$ ，计算 α_i 和中间残差 s 。若 s 已收敛，立即更新 x 并停止；否则继续求 $\hat{s}$ 、 $t = A\hat{s}$ ，计算 $\omega_i = t^\dagger s / (t^\dagger t)$ ，完成最终 r 和 x 更新。

补充参考 主要递推为

$$\beta_{i-1} = \frac{\alpha_{i-1} \rho_{i-1}}{\omega_{i-1} \rho_{i-2}}, \quad (5)$$

$$p^{(i)} = r^{(i-1)} + \beta_{i-1} \left(p^{(i-1)} - \omega_{i-1} v^{(i-1)} \right), \quad (6)$$

$$M\hat{p} = p^{(i)}, \quad v^{(i)} = A\hat{p}, \quad (7)$$

$$\alpha_i = \frac{\rho_{i-1}}{\hat{r}^\dagger v^{(i)}}, \quad (8)$$

$$s = r^{(i-1)} - \alpha_i v^{(i)}, \quad (9)$$

$$\omega_i = \frac{t^\dagger s}{t^\dagger t}, \quad (10)$$

$$r^{(i)} = s - \omega_i t, \quad x^{(i)} = x^{(i-1)} + \alpha_i \hat{p} + \omega_i \hat{s}. \quad (11)$$

实现中保留粗层热启动，但检测到 **breakdown** 或坏系数时回退冷启动。标量保留在设备端，减少 D2H/H2D 和同步。

1.7 第 7 页 | PyQCU Strict 特化 (主稿约 30 秒)

主口播稿 Strict 路径把粗层算子写成 $D_l = X_l + H_l$ ，再做左预条件 $\hat{D}_l = X_l^{-1} D_l$ ，最后投影得到 $D_{l+1} = R_l \hat{D}_l P_l$ 。运行期保存 onsite X, X^{-1} 、轴链接 Y 和预条件后的 $\hat{Y}$ 。顺序很重要：先在 *onsite* 层做 X^{-1} ，再做 P/R ，不能交换。

补充参考 细层使用 compact target parity，粗层始终保留 full geometry。粗层自由度为 $E = N_s^c n_v = 2n_v$ ，不是把 fine spin 直接删成 3×3 link。运行资产可以写成

$$\mathcal{G}_\ell = \{V_\ell, X_{\ell+1}, X_{\ell+1}^{-1}, Y_{\ell+1}^{\text{f/b}}, \hat{Y}_{\ell+1}^{\text{f/b}}\}.$$

MATPC 与 full operator、Clover-PC 和 coarse-PC 必须使用一致语义。原始 Y 默认只用于 setup/诊断；正式求解优先驻留 $\hat{Y}$ 、 X 与 X^{-1} 。

1.8 第 8 页 | 粗层 X/Y 公式 (主稿约 35 秒)

主口播稿 这一页专门说明粗层 X 与 Y 。 X 是 coarse onsite 的 $E \times E$ dense block， Y_μ 是跨 aggregate 的方向链接，它们都不是 $SU(3)$ 矩阵。预条件链接中，forward 使用 $X^{-1} Y^{\text{f}}$ ，backward 使用 $Y^{\text{b}} X^{-\dagger}$ ，左右顺序和 backward site 都不能交换。

补充参考 定义

$$X_C(X) = \sum_{x \in \mathcal{A}_X} V_X^\dagger C V_X, \quad (12)$$

$$Y_\mu^{\text{f}}(X) = -\kappa \sum_{x \in \mathcal{A}_X} (A V_X)^\dagger U_\mu V_{X+\hat{\mu}}, \quad (13)$$

$$Y_\mu^{\text{b}}(X - \hat{\mu}) = -\kappa \sum_{x \in \mathcal{A}_X} V_X^\dagger U_\mu^\dagger A V_{X-\hat{\mu}}. \quad (14)$$

其中 $AV = C^{-1}V$ (Clover-PC)，raw path 取 $AV = V$ 。raw coarse action 为

$$(D_c z)(X) = X(X) z(X) + \sum_\mu \left[Y_\mu^{\text{f}}(X) z(X + \hat{\mu}) + Y_\mu^{\text{b}}(X - \hat{\mu})^\dagger z(X - \hat{\mu}) \right]. \quad (15)$$

预条件链接和 MATPC 为

$$\hat{Y}_\mu^f(X) = X^{-1}(X)Y_\mu^f(X), \quad (16)$$

$$\hat{Y}_\mu^b(X - \hat{\mu}) = Y_\mu^b(X - \hat{\mu})X^{-\dagger}(X), \quad (17)$$

$$M_p^c = I - \hat{H}_{pq}\hat{H}_{qp}. \quad (18)$$

PC dslash 只应用 $\hat{Y}$ hopping, 不能再乘一次 onsite X 。若把 $(X - \hat{\mu})^{-1}$ 放到错误位置, 非对易 dense block 会立即暴露错误, 因此验证必须覆盖方向、site、dagger 和左右顺序。

1.9 第 9 页 | V-cycle 与 FGMRES (主稿约 40 秒)

主口播稿 一次 MG 预条件器内部执行 pre-smoothing、restriction、递归粗层求解、prolongation 和 post-smoothing。最粗层使用 Bi-CGStab, 中间层递归。外层采用 flexible right-preconditioned GMRES, 每次 Arnoldi 列可以使用不同的非线性预条件器。restart 边界重新计算完整真残差。

补充参考 递归预条件器为

$$z_l = S_{\text{post}}^{\nu_2} [S_{\text{pre}}^{\nu_1}(r_l) + P_l M_{l+1}^{-1} R_l (r_l - A_l S_{\text{pre}}^{\nu_1}(r_l))]. \quad (19)$$

FGMRES 的右预条件列为

$$z_j = M^{-1}v_j, \quad w_j = Az_j,$$

并通过 modified Gram–Schmidt 构造 Hessenberg 矩阵。最细层迭代只记外层 FGMRES, 不把 smoother 和 coarse solver 迭代相加。若 Arnoldi 历史长度为 m , 融合工作区为 $(2m + 5)B_f + 2B_c$, 其中 B_f 是一个 compact fine 向量, B_c 是一个 full first-coarse 向量。

1.10 第 10 页 | CUDA-C++ 优化 (主稿约 40 秒)

主口播稿 CUDA-C++ 的优化分为四层。细层 kernel 处理 spin projection、 $SU(3)$ 重用和合并访存; 粗层通过 device scalar、融合标量更新和 CUDA Graph 减少 launch 与同步; setup 使用分块/着色 Galerkin、显存预算、持久 hierarchy 和校验缓存; 多 rank 通信使用 rank-local halo、非阻塞向量 halo 和远端 correction 融合。

补充参考

- complex64 distributed vector halo 默认启用 overlap;
- complex128 改变浮点累加顺序后可能显著改变收敛轨迹, 默认关闭;
- link halo 目前仍为阻塞式 Sendrecv, 但按 pointer identity 缓存;
- runtime cache 使用 schema v2, 逐 tensor SHA256 校验, 并以原子方式发布;
- setup 采用 colored/batched Galerkin, 避免全逐列探测;
- FGMRES workspace 在相同几何和 restart 下复用。

关键判断是: 细层是带宽瓶颈, 粗层是延迟瓶颈, 通信和 setup 又属于另一类瓶颈, 不能用同一优化方法处理。

1.11 第 11 页 | QUDA 对照口径（主稿约 30 秒）

主口播稿 性能对照严格采用 test27。请求矩阵是 72 个 combined，精确完成 66 个 combined、132 条 side record；其余 6 个双 P100 c64 大格请求项因显存或 570 秒 setup 时限被显式替代。每侧执行 1 次 cold、2 次 warmup、5 次 steady，正式性能使用 trace-off。

补充参考 所有精确单元满足：

- config hash 一致：66/66；
- input bundle 一致：66/66；
- full-operator 真残差 $< 5 \times 10^{-6}$ ：66/66；
- trace-on MG 收敛曲线：44/44；
- MG level row 完整：220/220。

trace-on 包含额外同步和记录，只用于阶段归因。容量替代项只能与替代格点比较，不能进入原大格的加速比。

1.12 第 12 页 | 总体结果（主稿约 35 秒）

主口播稿 定义 $R = t_{\text{QUDA}}/t_{\text{PyQCU}}$ ， $R > 1$ 表示 PyQCU 更快。66 个精确单元整体中位只有 1.025；但 MultiGrid 有 35/44 个单元有利，其中 trace-off MG 为 21/22。相对地，BiCGStab reference 为 0/22，QUDA 在所有精确点上更快。因此结论必须限定在 MG 场景。

补充参考 MG-2 为 19/22 个单元有利，MG-3 为 16/22。逐点比值范围约为 0.18–14.63，离散度很大。最大的 14.63 是单点，不是置信区间；V100 c128 MG-2 也存在 0.54 慢点。最可信的正式口径是 trace-off，而不能用 trace-on 的同步开销替代。

1.13 第 13 页 | 设备与精度趋势（主稿约 35 秒）

主口播稿 V100 上 MG-3 优势最大：c64 和 c128 的 trace-off 分组中位比分别为 4.74 和 3.22。双 P100 的 MG-2/3 中位数在 1.15 到 1.79 之间，表现为总体有利但幅度较小。BiCGStab 在全部设备上仍由 QUDA 领先。

补充参考

设备/精度	MG-2	MG-3	BiCGStab
V100/c64	2.57	4.74	0.28
V100/c128	2.03	3.22	0.26
P100×2/c64	1.78	1.79	0.34
P100×2/c128	1.25	1.15	0.23

解释时要注意：P100/c64 每组只有 $n = 2$ ，统计稳健性弱于其他组；极值对格点体积和残差轨迹敏感。

1.14 第 14 页 | 阶段、显存与容量 (主稿约 40 秒)

主口播稿 trace-on 阶段分解表明, PyQCU 的 coarse solve 和 other 合计约占 95%, 说明优化重点已经转向粗层同步和 setup, 而不是细层算术。容量方面, 双 P100 c64 大格三条路径约为 15.9、14.8 和 12.4 GB; CPU staging 虽然降低显存, 但 setup 仍超过 570 秒, 因此 6 个请求项被替代。

补充参考 最大 device-wide 观测显存为:

- PyQCU: 11907.3 MiB;
- QUDA: 23063.3 MiB。

该数值可能包括同设备上的其他进程, 用于门限检查, 不声明为占进程峰值。两层/三层和不同格点的内存必须按设备、精度、层数和 trace 模式分组。

1.15 第 15 页 | 残差、边界与下一步 (主稿约 35 秒)

主口播稿 正确性方面, PyQCU 和 QUDA 最大真残差分别为 7.40×10^{-7} 和 7.97×10^{-7} , 均低于 5×10^{-6} 门限。下一步优先降低 coarse solve 与同步成本, 同时改进 setup 显存、device-aware 通信和更多平台的端到端验证。

补充参考 当前可证实的优势是 MG 专项性能、V100 表现、较低最大显存和自主可控; 当前不足是总体中位优势有限、BiCGStab 较慢、trace-on 开销明显、P100 大格容量受限以及拓扑覆盖有限。不能宣称任意格点、任意层数或所有求解器普遍领先。

1.16 第 16 页 | 参考实现与复现 (主稿约 20 秒)

主口播稿 参考实现包括 quantum-mg、QUDA 和 DD- α AMG。复现顺序是构建、运行 Strict fast gate、运行双 rank MPI probe, 最后进入完整矩阵。

补充参考

- 环境: `source ./env.sh;`
- 构建: `bash ./build.sh` 与 `bash ./install.sh;`
- 快速门禁: `run_strict_fast.py --fail-fast;`
- MPI 真残差: `strict_mpi_solve_probe.py;`
- 单元对照: `bench_strict_vs_quda.py;`
- 完整矩阵: `bench_mg_matrix_full.py。`

正式运行必须记录库 SHA、CUDA architecture、config hash、input bundle 和 cache expect/miss/hit。

1.17 第 17 页 | 508 报告的大规模先前成果 (主稿约 25 秒)

主口播稿 这一页补充先前成果。508 报告在 4 卡 DCU 环境中记录了从 64 到 1024 进程的 TEST8 c64 大格测试; 1024 进程时报告累计加速比为 Wilson 3.12、Clover 7.19。右下图给出 Clover 与 Wilson 单次迭代耗时差随等效体积增长的数据。

补充参考 这是早期 DCU/CUDA 传播子求解成果，证明当时的正确性和大规模扩展趋势。1024 是进程/线程规模，报告中 64 卡池容量估算约为 2048 GiB，不能写成 1024 张 GPU 的已知聚合性能。报告中的 MG 仍属于前期验证版本，因此不能把 508 数据与 test27 的 Strict-MG 正式比值混合。当前公开强证据仍包括 V100 单卡、双 P100、1/2/4 rank preflight；更大规模 STRICT-MG 仍需独立端到端复验。

1.18 第 18 页 | 致谢 (主稿约 10 秒)

主口播稿 我的汇报到这里。核心结论是 MultiGrid 专项性能已经进入国际先进实现同量级，并在 V100 等场景形成清晰优势；下一步聚焦粗层同步、通信和 setup 显存。谢谢大家。

2 公式与数据速查

主题	关键公式或数据
Wilson 算子	$D_W = (m_0 + 4)I - \frac{1}{2}H = (m_0 + 4)(I - \kappa H)$, $\kappa = 1/(2m_0 + 8)$ 。
Clover onsite	$T_p = -\frac{\kappa_{\text{Csw}}}{8u_0} \sum_{\mu < \nu} \gamma_\mu \gamma_\nu C_{\mu\nu}$, $A_p = I_p + T_p$ 。
奇偶 Schur	$S_o = A_o - \kappa^2 H_{oe} A_e^{-1} H_{eo}$ 。
Galerkin	$R = P^\dagger$, $A_{l+1} = R_l A_l P_l$ 。
Strict 分解	$D_l = X_l + H_l$, $\hat{D}_l = X_l^{-1} D_l$, $D_{l+1} = R_l \hat{D}_l P_l$ 。
粗层 X	$X_C(X) = \sum_{x \in A_X} V_X^\dagger C V_X$ 。
粗层 Y	$Y_\mu^f(X) = -\kappa \sum (A V_X)^\dagger U_\mu V_{X+\hat{\mu}}$; backward 使用 $U^\dagger A V_{X-\hat{\mu}}$ 。
预条件链接	$\hat{Y}^f = X^{-1} Y^f$, $\hat{Y}^b = Y^b X^{-\dagger}$ 。
MATPC	$M_p^c = I - \hat{H}_{pq} \hat{H}_{qp}$ 。
FGMRES workspace	$(2m + 5)B_f + 2B_c$ 。
正式性能	66 精确单元; MG 35/44; trace-off MG 21/22; BiCGStab 0/22。
残差	PyQCU 7.40×10^{-7} ; QUDA 7.97×10^{-7} ; 门限 5×10^{-6} 。
显存	PyQCU 最大 11907.3 MiB; QUDA 最大 23063.3 MiB。
508 报告	TEST8 c64, 64–1024 进程; 报告加速比 Wilson 3.12、Clover 7.19。

3 问答题

1. **为什么 $m_0 + 4 = 1/(2\kappa)$?** 由 Wilson 归一化定义 $\kappa = 1/(2m_0 + 8)$ ，两边取倒数即可。这是检查不同求解器归一化是否一致的第一道门。
2. **为什么矩阵不能显式存储?** 粗矩阵维度为 EN_c ，稠密存储是 $O((EN_c)^2)$ 。代表性几何下 c64 已约 11.1 TB，运行期只能保存有限支撑 stencil/block。
3. **X 和 Y 有什么本质区别?** X 是 coarse onsite 的 dense $E \times E$ block; $Y_\mu^{f/b}$ 是跨 aggregate 的一跳方向链接。两者都不是 $SU(3)$ 链接。
4. **为什么 $\hat{Y}^b = Y^b X^{-\dagger}$?** backward 原始项为 $Y^b(X - \hat{\mu})^\dagger z(X - \hat{\mu})$ 。左乘 $X^{-1}(X)$ 后，为保持 dagger 形式，必须得到 $Y^b(X - \hat{\mu})X^{-\dagger}(X)$ 。
5. **外层为什么用 FGMRES?** MG 预条件器在每个 V-cycle 中可能变化，不是固定线性算子。Flexible GMRES 允许每一步使用不同预条件器。

6. **最粗层为什么用 Bi-CGStab ?** 粗层规模小但非正规, Bi-CGStab 不要求 Hermitian positive definite, 且比完整 GMRES 的存储更轻。
7. **为什么总中位只有 1.025 ?** 全矩阵包含 BiCGStab reference; MG 单独是 35/44, trace-off MG 为 21/22。总中位不能替代分层结论。
8. **为什么 trace-on 不能作为正式性能 ?** trace-on 引入额外同步与记录, 用于阶段归因, 不是生产路径。
9. **为什么 complex128 overlap 默认关闭 ?** 改变浮点累加顺序后, 大格三层曾有 32 次外迭代退化到 451 次的记录。
10. **508 报告是否证明当前 Strict-MG 的大规模扩展 ?** 不直接证明。它证明早期 DCU 上 64–1024 进程的 Wilson/Clover BiCGStab 大规模传播子求解能力; 当前 Strict-MG 公开强证据是 V100、双 P100 和 1/2/4 rank 验证。
11. **下一步最优先做什么 ?** coarse solve/残差检查同步融合、device-aware MPI、setup 流式化和国产平台端到端复验。

4 源码定位与实现附注

4.1 Python、Cython 与 CUDA-C++ 边界

Python 层负责格点几何、张量布局、null-vector 生成、层级构造、缓存身份和求解器编排。Cython 桥负责检查 shape、取张量指针, 并在 nogil 段进入 C ABI。C++ 层解析 params/argv/set_ptrs, 创建持久 hierarchy, 并组织 CUDA kernel 与 MPI 通信。不要把 Python tensor layout、canonical null-vector layout 和 C++ blocked ABI 混为同一个对象。

关键入口:

- Strict 生命周期与 C++ hierarchy: `cpp/cuda/qcu/src/apply_multigrid_strict.cu`;
- Cython/ABI: `pyqcu/cuda/qcu/qcu.pyx` 与 `qcu_api.pxd`;
- 粗层 Galerkin、support 与 X/Y/Yhat: `pyqcu/tools/_strict_galerkin.py`;
- Python V-cycle 与 diagnostics: `pyqcu/solver/_quda_multigrid.py`;
- runtime cache: `pyqcu/cuda/_strict_cache.py`。

4.2 Setup 与 cache 的经验

Galerkin 构造不是显式矩阵乘法, 而是通过 prolong–apply–restrict 探测生成粗层 block。为防止显存峰值失控, 实现使用分块、着色和 workspace 预算; 当显存不足时使用更保守的 batch。runtime cache 要求身份完全一致, 逐 tensor 校验 SHA256, 原子发布, 命中前验证 manifest 和 shape/dtype。正式 benchmark 应显式给出 cache directory 和 miss/hit 预期。

4.3 分布式实现边界

当前多 rank 粗层只交换活跃 face, 不执行全量 gather。complex64 vector halo 可以使用非阻塞通信与 local base kernel 重叠; complex128 因浮点累加顺序会改变收敛轨迹, 默认关闭该路径。link halo 仍使用阻塞 Sendrecv, 但按 pointer identity 缓存。device-aware MPI 和 NVSHMEM 是后续优化, 不应写成已完成的生产能力。

5 测试与复现清单

层级	检查内容
Tier 0	CPU 代数、导出符号、FGMRES 边界、几何守卫、R/P 伴随、MATPC、asset layout 和 Galerkin 模型。
Tier 1	小格 CUDA Strict primitive、V-cycle、完整求解、persistent workspace 复用、warm x_0 、预算守卫和 complex128 dispatch。
Tier 2	真实 gauge + canonical null vector + QUDA formal collector；要求 config/input hash、1 cold + 2 warmup + 5 steady、独立真残差、cache provenance 和 schema-v2 memory evidence。

正式对照前的最小流程：

1. 构建并记录 `libqcu.so` SHA 与 `cubin architecture`；
2. 运行 `run_strict_fast.py --fail-fast`；
3. 运行双 rank `strict_mpi_solve_probe.py`；
4. 检查 cache identity 和 miss/hit；
5. 运行单单元 fair collector，再进入完整矩阵；
6. 对 combined 结果检查两侧状态、hash、真残差和 `fair=true`。

6 历史经验与常见误区

6.1 版本演进要点

- 早期 MuST 与 legacy 路径证明了 coarse solve、P/R 和 halo 的关键性，但小格加速不能外推到大格；
- CUDA Graph 和设备端标量显著降低 tiny-kernel 与同步成本，但算法本身缺少谱分离时仍可能没有端到端收益；
- Strict 路径将 full coarse $X/Y/\hat{Y}$ 、MATPC 和真残差语义拆开，解决了早期 Schur/full operator 混淆问题；
- 分布式 Strict 增加 halo、全局归约和 R/P 边界契约；c64 overlap 可用，而 c128 必须安全回退；
- test27 最终采用 capacity-aware 口径，明确记录 6 个替代项，比把缺失点静默填满更可信。

6.2 容易混淆的对象

- raw Wilson H 与完整 Wilson 算子 D_W ；
- fine Clover onsite A_p 与 coarse onsite X ；
- raw coarse link Y 与预条件链接 $\hat{Y}$ ；
- full coarse field 与 compact target-parity field；
- trace-on 诊断时间与 trace-off 正式性能；
- 容量替代格点与原始请求格点；
- 508 报告的早期大规模成果与 test27 的当前 Strict-MG 数据。

6.3 若时间充足的补充说明

可以进一步解释 $1 \pm \gamma_\mu$ 的 spin projector 结构, 为什么 coarse $E = 2n_v$, 为什么 Clover-PC 需要双向链接, 为什么 backward link 必须取 $X - \hat{\mu}$ 的 storage block, 以及为什么 device scalar 能降低粗层同步。若听众关注 GPU 性能, 可以补充 CUDA Graph、cooperative fused coarse solve、staged halo 和 ‘workspace cap’ 的设计取舍。若听众关注国产平台, 应把 508 报告作为早期成果、当前 CUDA 证据作为现状、DCU/CANN 真机验证作为下一步。